

Árvores e Suas Generalizações

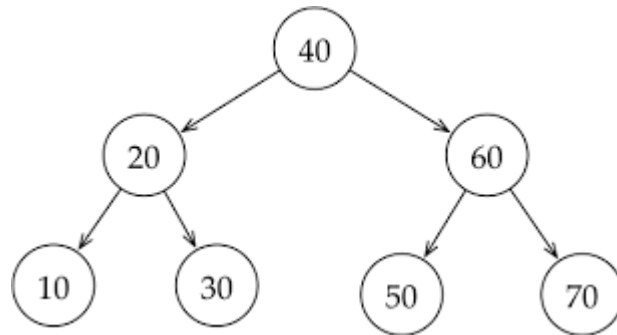
Aula 05 – Árvores Binárias de Busca: Inserção e Remoção



Árvores Binárias de Busca - ABB

Árvores Binárias de Busca - ABB

- **Não** permite chaves repetidas;
- Elementos da sub-árvore da esquerda são menores que o valor do nó;
- Elementos da sub-árvore da direita são maiores que o valor do nó;
- Todas as sub-árvores também são árvores binária de busca.



Árvores Binárias de Busca - ABB

- Monte a Árvore Binária de Busca para os elementos: 6, 2, 4, 1, 9, 8, 12, 19, 5, 3, 20

ABB – Inserindo novo nó

ABB Inserindo novo nó

- A inserção segue a lógica da busca:
- Se a árvore estiver vazia, o novo nó se torna a raiz.
- Se o valor a ser inserido for menor que o nó atual, ele é direcionado para a sub-árvore esquerda.
- Se for maior, ele é direcionado para a sub-árvore direita.
- O processo se repete até encontrar uma posição vazia onde o novo nó pode ser inserido.

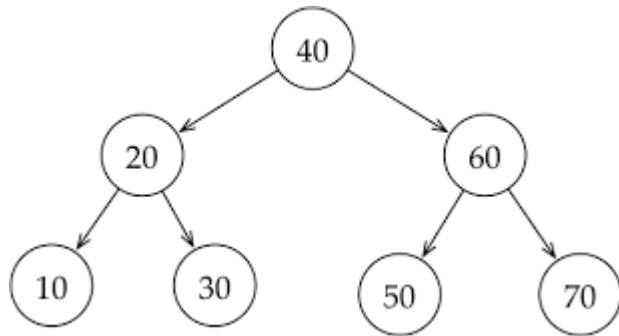


ABB – Inserindo novo nó

// Função para criar um novo nó

```
Node* novoNo(int valor){  
  
    Node* node = (struct Node*) malloc(sizeof(struct Node));  
  
    node->valor = valor;  
  
    node->esquerda = node->direita = NULL;  
  
    return node;  
}
```

// Função para inserir um valor na árvore binária de busca

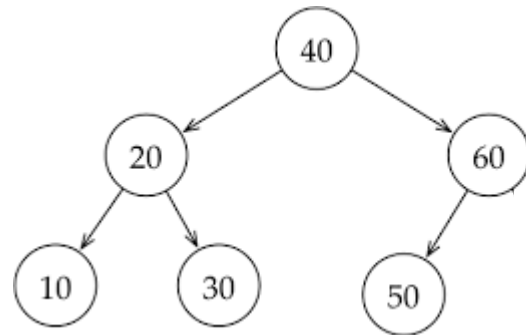
```
struct Node* inserir(struct Node* raiz, int valor){  
  
    // Se a árvore estiver vazia, cria um novo nó  
    if (raiz == NULL)  
        return novoNo(valor);  
  
    // Se o valor for menor, insere na sub-árvore esquerda  
    if (valor < raiz->valor)  
        raiz->esquerda = inserir(raiz->esquerda, valor);  
  
    // Se o valor for maior, insere na sub-árvore direita  
    else if (valor > raiz->valor)  
        raiz->direita = inserir(raiz->direita, valor);  
  
    return raiz; // Retorna a raiz atualizada  
}
```

ABB – Removendo nó

Removendo nó

A remoção pode envolver três casos:

1. O nó não tem filhos → Ele é simplesmente removido.
2. O nó tem um filho → O filho substitui o nó removido.
3. O nó tem dois filhos → O nó é substituído pelo seu sucessor imediato (o menor valor da sub-árvore direita).



Removendo nó

```
// Função para encontrar o menor valor em uma sub-árvore
Node* encontrarMinimo(struct Node* node) {
// O menor valor está na extrema esquerda
while (node->esquerda != NULL)
    node = node->esquerda;
return node;
}
```

```
// Função para remover um valor da árvore
Node* remover(struct Node* raiz, int valor) {
    if (raiz == NULL)
        return raiz; // Caso base: árvore vazia
    // Se o valor for menor, busca na sub-árvore esquerda
    if (valor < raiz->valor)
        raiz->esquerda = remover(raiz->esquerda, valor);
    // Se o valor for maior, busca na sub-árvore direita
    else if (valor > raiz->valor)
        raiz->direita = remover(raiz->direita, valor);
    else {
        // Caso 1: Nó sem filhos
        if (raiz->esquerda == NULL && raiz->direita == NULL) {
            free(raiz);
            return NULL;
        }
    }
}
```

```
// Caso 2: Nó com um filho
else if (raiz->esquerda == NULL) {
    struct Node* temp = raiz->direita;
    free(raiz);
    return temp;
} else if (raiz->direita == NULL) {
    struct Node* temp = raiz->esquerda;
    free(raiz);
    return temp;
}
// Caso 3: Nó com dois filhos
// Encontra o sucessor
Node* temp = encontrarMinimo(raiz->direita);
// Substitui o valor
raiz->valor = temp->valor;
// Remove o sucessor
raiz->direita = remover(raiz->direita, temp->valor);
}
// Retorna a raiz atualizada
return raiz;
}
```

Exercícios ABB

Inserção em Árvore Binária de Busca

- 1) Insira os seguintes valores em uma árvore binária de busca: 50, 30, 70, 20, 40, 60, 80
 - a) Desenhe a árvore resultante.
 - b) Implemente o código de inserção.
 - c) Mostre a árvore em ordem crescente.

Exercícios ABB

Remoção em Árvore Binária

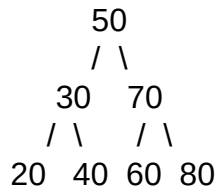
2) Usando a árvore do exercício anterior:

- a) Remova o nó 20 (sem filhos)
- b) Remova o nó 30 (com um filho)
- c) Remova o nó 50 (com dois filhos)
- d) Desenhe a árvore após cada remoção.
- e) Mostre a árvore em ordem após todas as remoções.

Exercícios ABB

Busca em Árvore Binária

3) Com a árvore abaixo:

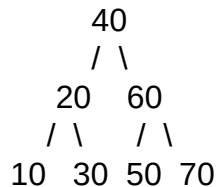


Implemente um algoritmo em C/C++ que busque os valores: 60, 25, 40. Indique se foram encontrados.

Exercícios ABB

Travessias da Árvore

4) Com a árvore abaixo:



Escreva as travessias: (apenas o print do resultado final)

- a) Pré-ordem
- b) Em-ordem
- c) Pós-ordem

Exercícios ABB

Desafio: Menu interativo em C/C++

Crie um menu com as opções:

1. Inserir valor
2. Remover valor
3. Buscar valor
4. Exibir em ordem

Dica: Use um laço while com switch-case.



C . E . S . A . R

Pessoas impulsionando inovação.
Inovação impulsionando negócios.

